

Lab 1: Introduction to Beetlebot and ROS2

Commands

Part 2

Objectives

- Publish and Subscribe Topics
- Teleoperate Robot
- RViz

ROS2 Workspace Creation

```
mkdir -p ~/beetlebot_ws/src
```

```
cd ~/beetlebot_ws
```

```
colcon build
```

```
source install/setup.bash
```

Check ROS2 installation and Display available topics

ros2 doctor

ros2 topic list

ros2 node list → Display robot information

rviz2

Launch rviz

```
rviz2 -d ~/beetlebot_ws/src/beetlebot_description/rviz/beetlebot.rviz
```

Robot Description

```
ros2 topic echo /robot_description
```

Display TF Tree

```
ros2 run tf2_tools view_frames
```

Monitor Transform Data

```
ros2 run tf2_ros tf2_echo odom base_link
```

Display Robot Pose

```
ros2 topic echo /odom
```

Display Laser Scan

```
ros2 topic echo /scan
```

Display Camera Images

```
ros2 topic info /camera/image_raw
```

View IMU Data

```
ros2 topic echo /odom
```

Display Point Cloud

```
ros2 topic echo /points
```

Display Occupancy Grid Map

`ros2 topic echo /map`

Display Planned Path

`ros2 topic echo /plan`

Display Point Cloud

`ros2 topic echo /points`

- ROS 2 is a middleware that enables communication between different software modules (called nodes) in a robot.
- ROS 2 divides the application into multiple nodes. Each node performs a specific task.
- Example:
 - Camera Node captures images.
 - LiDAR Node reads obstacle distances. Navigation Node plans the path.
 - Motor Controller Node drives the motors

- These nodes communicate through Topics.
- A Publisher sends information to a topic.
- A Subscriber receives information from the topic.

ROS2 Publishing – Understanding Twist Messages

- Twist
 - Linear Velocity → Moves robot in straight direction
 - linear.x → +ve : Forward, -ve : Backward
 - Angular Velocity → Rotates the robot
 - angular.z → +ve : Left, -ve : Right

- `ros2 pkg create beetlebot_control --build-type ament_python --dependencies rclpy geometry_msgs`
- `cd beetlebot_control/beetlebot_control`
- `Touch/gedit move_forward.py`
- `chmod +x move_forward.py`

Move_forward.py → Python Code

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
class MoveRobot(Node):
    def __init__(self):
        super().__init__('move_robot')
        self.publisher = self.create_publisher(Twist,
        '/cmd_vel', 10)
        self.timer=self.create_timer(0.1,self.publish_velocity)
        self.get_logger().info("Moving Forward")
    def publish_velocity(self):
        velocity=Twist()
        velocity.linear.x=0.2
        velocity.linear.y=0
        velocity.linear.z=0
```

```
        velocity.angular.x=0.0
        velocity.angular.y=0.0
        velocity.angular.z=0.0
        self.publisher.publish(velocity)
        self.get_logger().info("Moving Forward")

def main(args=None):
    rclpy.init(args=args) # Create
    Node node = MoveForward()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info("Program
        Interrupted")
    finally:
        stop = Twist()
        stop.linear.x = 0.0
        stop.linear.y = 0.0
```

Move_forward.py → Python Code

```
stop = Twist()  
stop.linear.x = 0.0  
stop.linear.y = 0.0  
stop.linear.z = 0.0  
stop.angular.x = 0.0  
stop.angular.y = 0.0  
stop.angular.z = 0.0  
node.publisher_.publish(stop) node.get_logger().info(  
    "Robot Stopped")  
Nodenode.destroy_node()  
rclpy.shutdown()
```

```
if __name__ == '__main__':  
    main()
```

Execute the code

```
cd ~/beetlebot_ws  
colcon build  
source install/setup.bash  
ros2 pkg list | grep beetlebot_control  
ros2 run beetle_control move_forward
```

To verify if the python is working:

```
python3 -c "import beetlebot_control.move_forward;  
print('OK')"
```

If “armed” is not True, then given the below commands

```
ros2 service call /lyra/arm std_srvs/srv/Trigger
ros2 topic echo armed
```

To Stop temporarily:

```
ros2 service call /lyra/disarm std_srvs/srv/Trigger
```

To stop permanently without disarm:

```
ros2 topic pub -r 20 -t 40 -once /cmd_vel_nav geometry_msgs/msg/Twist
“{linear:x:0.0,y:0.0,z:0.0}, {angular:x:0.0,y:0.0,z:0.0}}”
```

//in the above command -r 20 and -t 40 can be removed if you don't want to force this command

```
ros2 topic pub -once /cmd_vel_nav geometry_msgs/msg/Twist “{linear:x:0.2,y:0.0,z:0.0},
{angular:x:0.0,y:0.0,z:0.0}}”
```

Ros2 run teleop_twist_keyboard teleop_twist_keyboard